

# PDF → JSON Project — College Reference Guide

**Purpose:** Practical printout for the current working project: exact structure, current code, new-PDF procedure, backups, and code-change rules.

## 1. CURRENT PROJECT STRUCTURE

Keep the project arranged like this:

```
samarth/  
■■■ main.py  
■■■ pdf_parser.py  
■■■ input/  
■   ■■■ input.pdf  
■■■ output.json
```

**main.py** — opens the PDF, reads pages, separates procedures, calls the parser, creates JSON, and adds PDF information.

**pdf\_parser.py** — extracts Procedure, Time, Dur, and status/STEP IDs.

**input/input.pdf** — the one PDF processed by the current program.

**output.json** — generated result; overwritten on the next run.

## 2. CURRENT WORKING CODE — main.py

```
import fitz  
import re  
import json  
  
from pdf_parser import extract_process_info, extract_status  
  
pdf = fitz.open("input/input.pdf")  
all_text = ""  
pages_read = 0  
  
for page in pdf:  
    all_text = all_text + page.get_text() + "\n"  
    pages_read = pages_read + 1  
  
matches = list(  
    re.finditer(  
        r"Procedure\s*:",  
        all_text,  
        re.IGNORECASE  
    )  
)  
  
procedure_blocks = []  
  
for i in range(len(matches)):  
    start = matches[i].start()  
    if i + 1 < len(matches):  
        end = matches[i + 1].start()  
    else:  
        end = len(all_text)  
    block = all_text[start:end].strip()  
    procedure_blocks.append(block)  
  
all_processes = []  
for block in procedure_blocks:  
    Procedure, Time, Dur = extract_process_info(block)  
    status_data = extract_status(block)  
    process_data = {  
        "Procedure": Procedure,  
        "Time": Time,  
        "Dur": Dur,  
        "status": status_data  
    }  
    all_processes.append(process_data)  
  
pdf_info = {  
    "PDFInfo": {  
        "No. of Pages read": pages_read  
    }  
}  
all_processes.append(pdf_info)  
  
with open("output.json", "w") as file:  
    json.dump(all_processes, file, indent=4)  
  
print("Total pages:", pages_read)
```

```

print("Total text length:", len(all_text))
print("Total procedures:", len(procedure_blocks))
print("Total dictionaries created:", len(all_processes) - 1)
print("No. of Pages read:", pages_read)
print("JSON file created successfully.")

pdf.close()

```

### 3. CURRENT WORKING CODE — pdf\_parser.py

```

import re

def extract_process_info(text):
    Procedure = ""
    Time = ""
    Dur = ""

    match = re.search(
        r"Procedure\s*:\s*(.*?)\s*(?=Time\s*:\s|$)",
        text,
        re.IGNORECASE | re.MULTILINE
    )
    if match:
        Procedure = match.group(1).strip()

    match = re.search(
        r"Time\s*:\s*(.*?)\s*(?=Mode\s*:\s|$)",
        text,
        re.IGNORECASE | re.MULTILINE
    )
    if match:
        Time = match.group(1).strip()

    match = re.search(
        r"Dur\s*:\s*(.*?)\s*(?=Priority\s*:\s|$)",
        text,
        re.IGNORECASE | re.MULTILINE
    )
    if match:
        Dur = match.group(1).strip()

    return Procedure, Time, Dur

def extract_status(text):
    status_data = {}
    lines = text.splitlines()
    current_step_id = None

    for line in lines:
        line = line.strip()
        step_match = re.match(r"^\d+\.\d+\b", line)
        if step_match:
            current_step_id = step_match.group(1)
            continue

        status_match = re.search(
            r"\b(Success|Failure)\b",
            line,
            re.IGNORECASE
        )
        if status_match and current_step_id:
            status = status_match.group(1).lower()
            if status not in status_data:
                status_data[status] = []
            status_data[status].append(current_step_id)
            current_step_id = None

    return status_data

```

**Current status support:** Success and Failure. Do not add other statuses until the actual college PDF shows they are needed.

### 4. CURRENT JSON OUTPUT

Each procedure becomes one dictionary:

```

{
  "Procedure": "abbnm",
  "Time": "31-dec-2025 11:44:13.103",
  "Dur": "203.96(s)",
  "status": {
    "success": ["01.0001"],

```

```

        "failure": ["01.0002"]
    }
}

```

At the end:

```

{
  "PDFInfo": {
    "No. of Pages read": 6479
  }
}

```

## 5. WHEN YOU GET A NEW PDF — EXACT STEPS

**Important:** The current program processes one PDF at a time and opens **input/input.pdf**.

1. Keep a backup of the current working project.
2. Get the new PDF.
3. Open the project **input** folder.
4. Make the PDF you want to process the file named **input.pdf**.
5. Other PDFs may stay there with different names, but main.py only reads **input.pdf**.
6. Do not change main.py just because the PDF is new, if it follows the same supported structure.
7. Run **main.py** in PyCharm.
8. Wait for processing to finish.
9. Check the console counts.
10. Open **output.json** and inspect it.
11. If you want to preserve the result, copy/rename output.json before another run because the next run overwrites it.

Example:

```

input/
■■■ old_college_pdf.pdf
■■■ test_pdf.pdf
■■■ input.pdf      ← current PDF

```

Typical console output:

```

Total pages: 6479
Total text length: 11334398
Total procedures: 1254
Total dictionaries created: 1254
No. of Pages read: 6479
JSON file created successfully.

```

## 6. BACKUP / SAFE WORKING RULES

**Known-good version:** Before a new experiment or code change, keep a copy of the current project. Make changes in the working copy, run the program, check the console and output.json, and keep the update only if it works. If it fails, return to the known-good backup. Also preserve important test PDFs and successful JSON outputs separately.

## 7. IF A NEW STATUS IS FOUND — EXACT CODE-CHANGE PROCESS

If the actual college PDF contains **Progress**, **Suspend**, or another status, change only the status pattern in **pdf\_parser.py**.

**Step 1:** Open pdf\_parser.py.

**Step 2:** Find:

```

status_match = re.search(
    r"\b(Success|Failure)\b",
    line,
    re.IGNORECASE
)

```

**Step 3:** For Progress and Suspend, change it to:

```

status_match = re.search(
    r"\b(Success|Progress|Suspend|Failure)\b",
    line,
    re.IGNORECASE
)

```

**Step 4:** Save pdf\_parser.py.

**Step 5:** Run main.py with the PDF as input/input.pdf.

**Step 6:** Check output.json. The new status automatically becomes its own lowercase key with its STEP IDs.

```

"status": {
  "success": ["01.0001"],
  "progress": ["01.0002"],
  "suspend": ["01.0003"],

```

```
    "failure": ["01.0004"]  
}
```

**No main.py change is required just to add a status.**

For another status such as Pending, add it to the same pattern: **Success|Progress|Suspend|Failure|Pending**.

**Capitalization:** re.IGNORECASE already handles Success/SUCCESS/success.

**Important:** Do not add possible statuses in advance. Inspect the actual PDF first.

## 8. GENERAL RULES FOR NEW PDFS / NEW REQUIREMENTS

**Same structure, new PDF:** Replace input.pdf and run; usually no code change.

**New status:** Update the status regex in pdf\_parser.py.

**Different metadata layout:** Inspect extracted text first, then modify the relevant parser rule.

**Different STEP ID format:** Modify the STEP-ID regex only if the actual PDF requires it.

**New JSON field:** Add extraction logic in the parser and then add the field to process\_data in main.py.

**Multiple PDFs automatically:** New requirement; current project is one active input.pdf at a time.

**Very large PDF:** No fixed page-count rule, but larger PDFs can take more time and memory because all extracted text is stored in all\_text.

**Different overall structure:** Do not assume compatibility; inspect the PDF text before changing code.

## 9. PROJECT REMINDERS

- Non-AI: normal Python + PyMuPDF + regex + JSON.
- Working environment: Python 3.6.5 and PyMuPDF 1.18.18.
- Text extraction, not OCR.
- Designed for the known process/table PDF structure.
- STEP IDs such as 01.0001 remain strings.
- Mode, Source, Priority, and table TIME are intentionally ignored.

Use this as the college reference. When a real new PDF or requirement arrives, inspect it first and make the smallest necessary change.